

Bases de Datos a Gran Escala

Sesión 1 · Modelado de datos, modelo relacional y consultas SQL

Diego Sevilla Ruiz

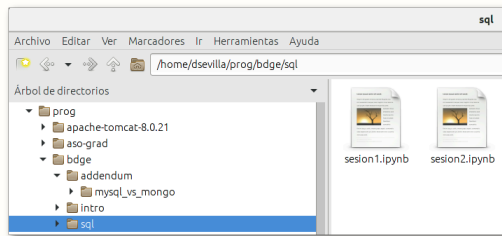
Departamento de Ingeniería y Tecnología de Computadores

Facultad de Informática · Universidad de Murcia

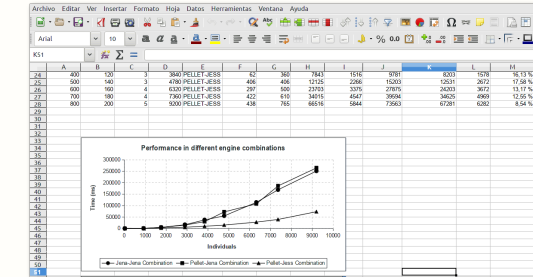
Modelado de datos

De una necesidad de información a una representación que podemos consultar y mantener.

¿Cómo se almacenan los datos?

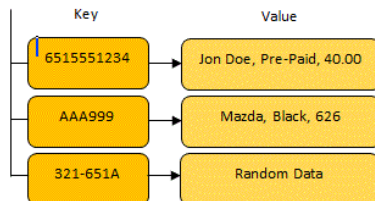


```
Id,PostId,CreationDate,BountyAmount,U
1,1,2015-10-29T00:00:00.000,,2
2,2,2015-10-29T00:00:00.000,,2
3,3,2015-10-30T00:00:00.000,,2
4,5,2015-10-30T00:00:00.000,,2
5,7,2015-10-30T00:00:00.000,,2
6,8,2015-10-30T00:00:00.000,,25,5
7,3,2015-10-30T00:00:00.000,,16
8,4,2015-10-30T00:00:00.000,,2
9,1,2015-10-30T00:00:00.000,,2
```



Directorios y ficheros. CSV y hojas de cálculo. Documentos tabulares.

También: pares clave/valor, registros, *streams* multimedia y formatos columnares.



¿Qué es un modelo de datos?

Todo modelo de datos define tres componentes:

- 1. Estructuras** para albergar los datos.
- 2. Restricciones** que deben cumplir.
- 3. Operaciones** que se pueden realizar.

Ejemplos de los tres componentes

Componente	Preguntas que responde
Estructuras	¿Filas y columnas? ¿Vértices y aristas? ¿Pares clave/valor? ¿Una secuencia de bytes?
Restricciones	¿Todas las filas tienen las mismas columnas? ¿Qué valores admite un atributo? ¿Puede un archivo tener dos padres?
Operaciones	¿Cuál es el valor de la clave <code>x</code> ? ¿Qué filas contienen «plaza»? ¿Cuáles son los siguientes N registros?

¿Por qué modelar?

- Describe un problema en el **nivel de abstracción correcto**
- Es una simplificación que conserva las características esenciales para resolverlo
- Facilita la comprensión y la comunicación entre personas y sistemas
- Hace explícitas las reglas del dominio
- Permite evolucionar la aplicación y razonar sobre sus costes

a datos crudos

The diagram shows a database schema with the following tables and attributes:

- Table 1:**
 - Attributes: AboutMe TEXT, AccountId INT(11)
 - Indexes: Indexes
- Table 2:**
 - Attributes: Id INT(11) (Primary Key), TagName TEXT, Count INT(11), WikiPostId INT(11), BlogPostId INT(11)
 - Indexes: Indexes
- Table 3:**
 - Attributes: Score INT(11), OwnerDisplayName TEXT, PostTypeId INT(11), OwnerUserId INT(11), Tags TEXT, CreationDate TIMESTAMP, FavoriteCount INT(11), CommunityOwnedDate TIMESTAMP
 - Indexes: Indexes

Relationships are indicated by lines connecting the tables.



```
-- Schema stackoverflow
-- Schema stackoverflow

CREATE SCHEMA IF NOT EXISTS stackoverflow DEFAULT CHARACTER SET utf8;
USE stackoverflow ;

-- Table 'stackoverflow' . Users

CREATE TABLE IF NOT EXISTS 'stackoverflow' . Users (
  id INT(1) NOT NULL DEFAULT 0,
  Displayname TEXT NULL DEFAULT NULL,
  Views INT(3) NULL DEFAULT NULL,
  Downvotes INT(1) NULL DEFAULT NULL,
  Upvotes INT(1) NULL DEFAULT NULL,
  Postflagging TEXT NULL DEFAULT NULL,
  Reputation INT(1) NULL DEFAULT NULL,
  Location TEXT NULL DEFAULT NULL,
  Age INT(1) NULL DEFAULT NULL,
  Unlocks INT(3) NULL DEFAULT NULL,
  Creationdate TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  Aboutme TEXT NULL DEFAULT NULL,
  Accountid INT(1) NULL DEFAULT NULL,
  PRIMARY KEY (id))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8;
```


Modelo relacional

Tablas, claves, restricciones y álgebra para consultar sin depender de cómo se almacenan físicamente los datos.

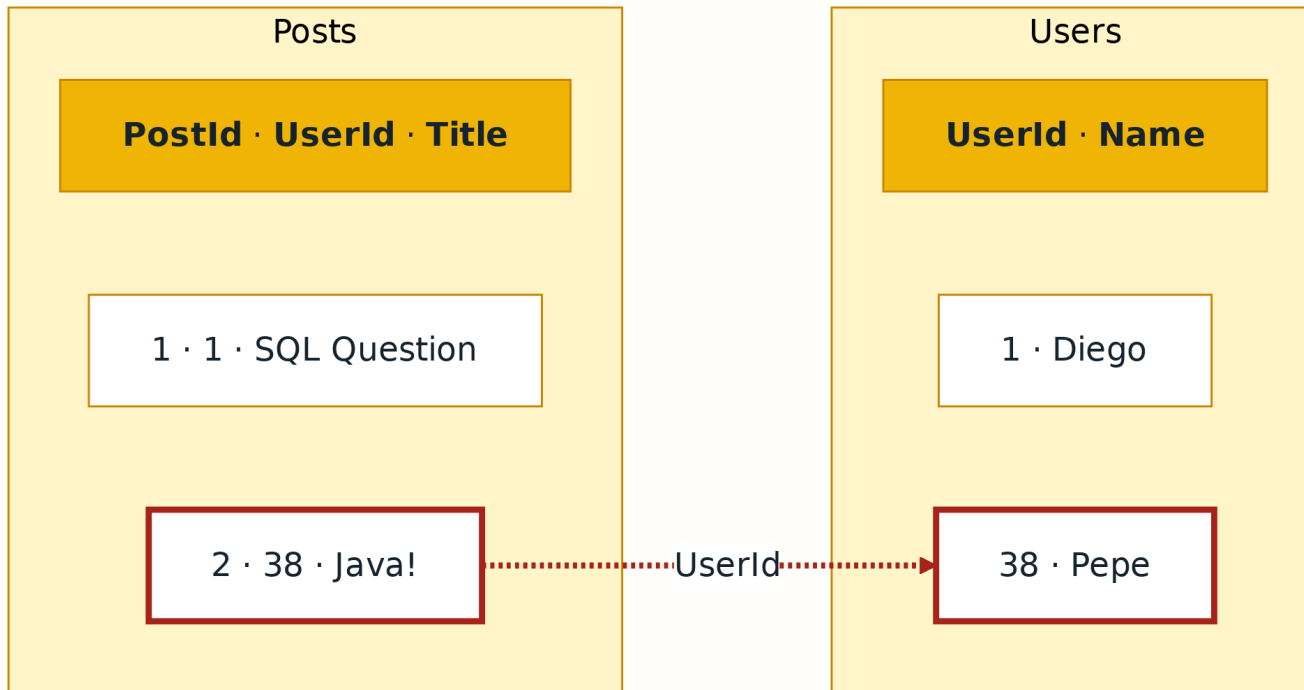
Modelo relacional

- Propuesto por **E. F. Codd en 1970**
- Modela los datos mediante **relaciones**, habitualmente representadas como tablas
- Se apoya en teoría de conjuntos, lógica de predicados y álgebra relacional
- Su fuerza es la independencia entre los datos y las aplicaciones que los usan: permite formular consultas no previstas al diseñar la base de datos

Conceptos esenciales

- Una relación tiene un **esquema**: atributos con nombre y dominio
- Cada fila o **tupla** sigue ese mismo esquema; cada atributo tiene su propio tipo o dominio
- Una **clave primaria** identifica de forma única cada fila y no admite `NULL`
- Una **clave ajena** expresa una referencia a una fila de otra tabla
- La normalización busca evitar redundancias y anomalías; la desnormalización es una decisión física posterior y consciente

Clave primaria y clave ajena



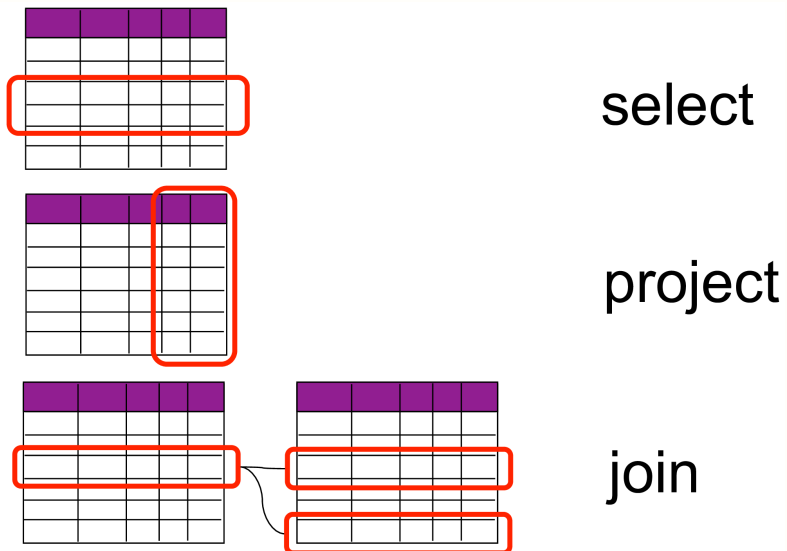
`Posts.UserId` referencia `Users.UserId` : no debería existir un *post* cuyo usuario no exista.

Restricciones de integridad

Restricción	Regla
Clave / entidad	No hay dos filas con la misma clave primaria; su valor no es nulo.
Dominio	Cada atributo solo contiene valores de su tipo y reglas asociadas.
Referencial	Una clave ajena apunta a una fila existente, salvo que se permita NULL .
De negocio	Reglas propias del dominio: importe positivo, fecha de fin posterior al inicio, etc.

Álgebra relacional

- Nos permite abstraernos de la representación física
- Las operaciones producen nuevas relaciones: se pueden componer y optimizar
- Operaciones que veremos en SQL: selección, proyección, producto cartesiano, unión y reunión (*join*)



Consultas equivalentes, costes diferentes

El optimizador puede reordenar operaciones equivalentes para reducir el volumen de datos intermedio.

```

$$\sigma_{p=\text{conoce}}(R) \bowtie_{o=s} (\sigma_{p=\text{tieneCuenta}}(R) \bowtie_{o=s} \sigma_{p=\text{web}}(R))$$

$$(\sigma_{p=\text{conoce}}(R) \bowtie_{o=s} \sigma_{p=\text{tieneCuenta}}(R)) \bowtie_{o=s} \sigma_{p=\text{web}}(R)$$

$$\sigma_{p1=\text{conoce} \wedge p2=\text{tieneCuenta} \wedge p3=\text{web}}(R \times R \times R)$$

```

No todas tienen el mismo coste, aunque describan el mismo resultado.

SQL: consultar datos

El lenguaje declarativo para expresar qué resultado queremos; el SGBD decide cómo obtenerlo.

SQL y sus dialectos

- SQL es un estándar ANSI/ISO; SQL:2023 incorpora, entre otras extensiones, consultas de grafos de propiedades
- **DML**: consulta y modificación de datos. **DDL**: definición del esquema
- SQL trabaja con tablas, columnas y filas; a diferencia del álgebra relacional, una consulta puede devolver duplicados (*bag semantics*)
- En esta sesión usamos el núcleo común de SQL. Cuando aparezca una diferencia, se indicará el motor: **SQLite**, **MySQL** o **DuckDB**

Herramientas de la asignatura

Herramienta	Papel
SQLite	Primer contacto: un único fichero, consola sencilla y sin servidor.
MySQL	Prácticas con servidor, transacciones, índices y planes <code>EXPLAIN</code> .
DuckDB	Análisis de datos y lectura directa de Parquet; útil para contrastar optimizaciones analíticas.

Los ejemplos deben ejecutarse con el motor indicado: la sintaxis y, sobre todo, los planes no son idénticos.

Modo de trabajo

- El objetivo es manejar SQL con comodidad, no memorizar un catálogo de funciones
- Trabajaremos sobre una instantánea de datos de Stack Overflow cargada por los notebooks de la asignatura
- El mismo conjunto permite estudiar esquema, consultas, calidad de datos, relaciones, agregación y rendimiento
- El cuaderno prepara los datos desde Parquet; DuckDB puede consultarlos directamente y MySQL permite estudiarlos como base de datos de servidor

Primer contacto con SQLite

SQLite almacena una base de datos relacional en un fichero y añade comandos de administración al intérprete:

```
$ sqlite3 ejemplo.db
sqlite> .tables
sqlite> .schema Posts
sqlite> .open otro-fichero.db
sqlite> .help
```

Estos comandos no son SQL: son ayudas de la consola de SQLite.

Base de datos de Stack Overflow

Questions

Tags

Users

Badges

Unanswered

Ask Question

How to remove comma at end of last line in an array

0

▲▼★

I am using `sed '$s/,,$//'` to try to remove the comma at the end of the final line of an array, which has the opening and closing brackets on their lines (as shown below). This command `sed '$s/,,$//'` isn't working as intended.

If I do `sed '$s/,$//'` it will remove the closing array bracket, so I know I'm kind of on the right track....

```
[{"foo": "foofoo", "fuzz": ["fee, fi, fo, fum"]}, {"bar": "barbar", "fuzz": ["blah", "blah"]}]
```

this is for a bash script. please do not provide answer that uses perl.

bash

sed

share

edit

close

flag

edited 1 hour ago

asked 1 hour ago

Leahcim

8,207 ● 27 ● 100 ● 196

Is this supposed to be valid JSON? The thing producing it should be fixed to stop adding the incorrect comma. – chepner 1 hour ago

@chepner yes, it is. not sure how to do that (subject for another question...) – Leahcim 1 hour ago

add a comment

2 Answers

active

oldest

votes

▲▼★

There is no comma in last line so `'$s/,,$//'` will not work for `sed`.

1

Instead of `sed` you can use `awk`:

```
awk ' $1 == "]" {sub(/,$/, "", s)} NR>1 {print s} {s=$0} END{print s}' file
```

```
[{"foo": "foofoo", "fuzz": ["fee, fi, fo, fum"]}, {"bar": "barbar", "fuzz": ["blah", "blah"]}]
```

asked today

viewed 16 times

active today

Featured on Meta

New Navigation Release Candidate

Hot Meta Posts

8

Is it wrong to try to draw attention to a post I feel should be closed?

5

What if I think a deleted answer is worthwhile?

25

Syntax Highlighting for Neo4j / Cypher questions

Looking for a job?

Senior PHP Developer

Trovit

Barcelona, Spain

php

mysql

Systems Engineer

Brandcrumb

Barcelona, Spain

hadoop

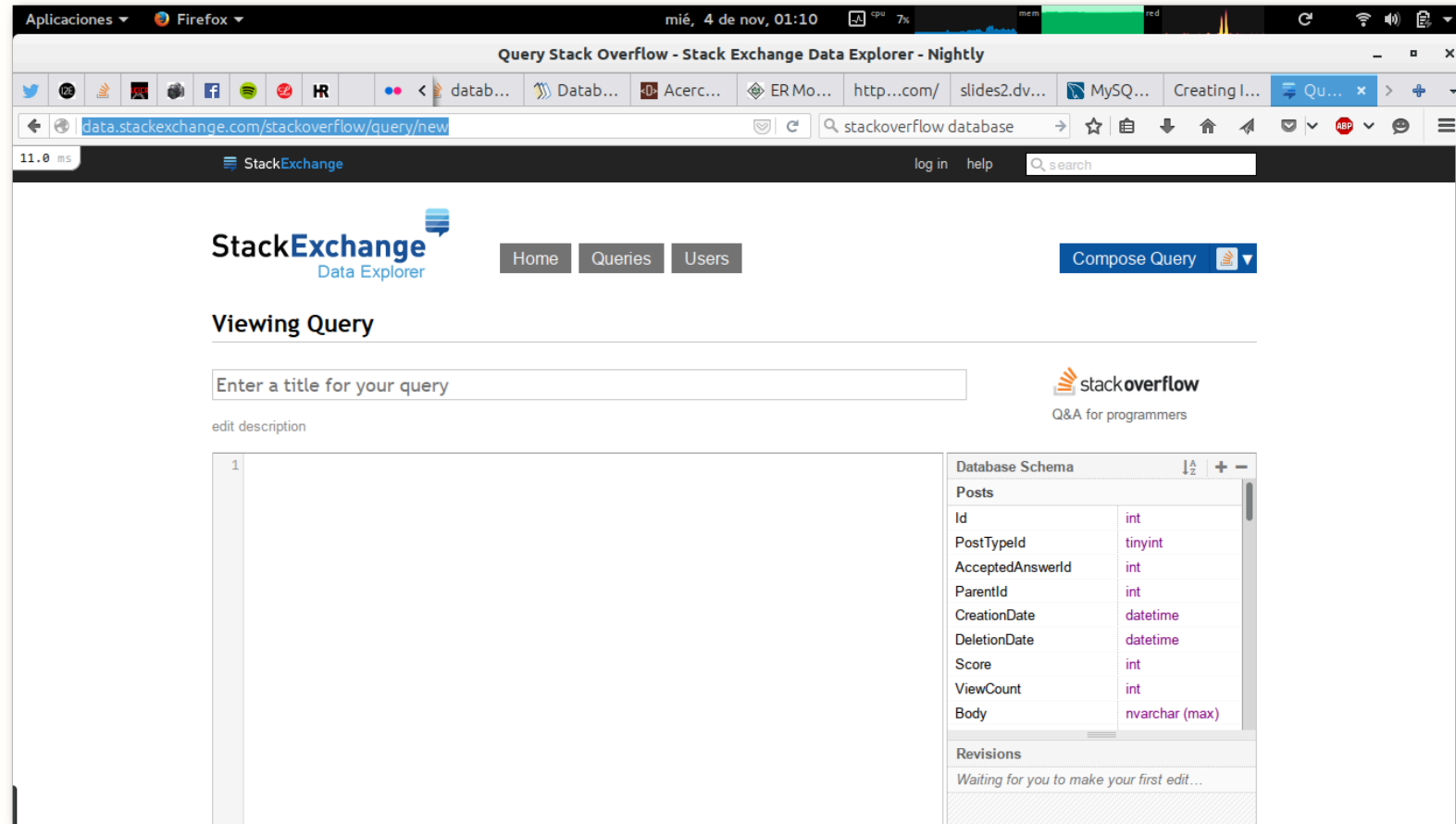
nosql

Related

190

How can I remove the first

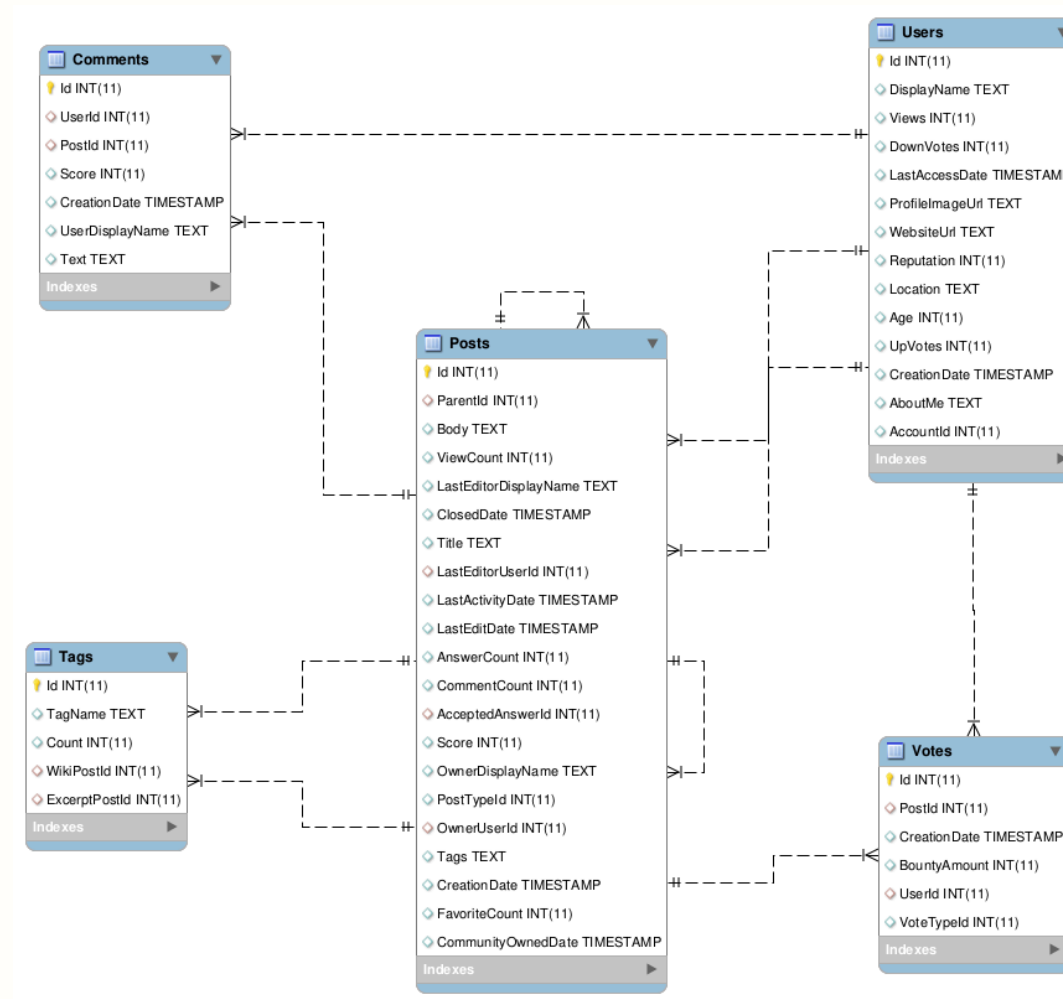
Exploración de datos en Stack Exchange Data Explorer



Esquema de Stack Overflow

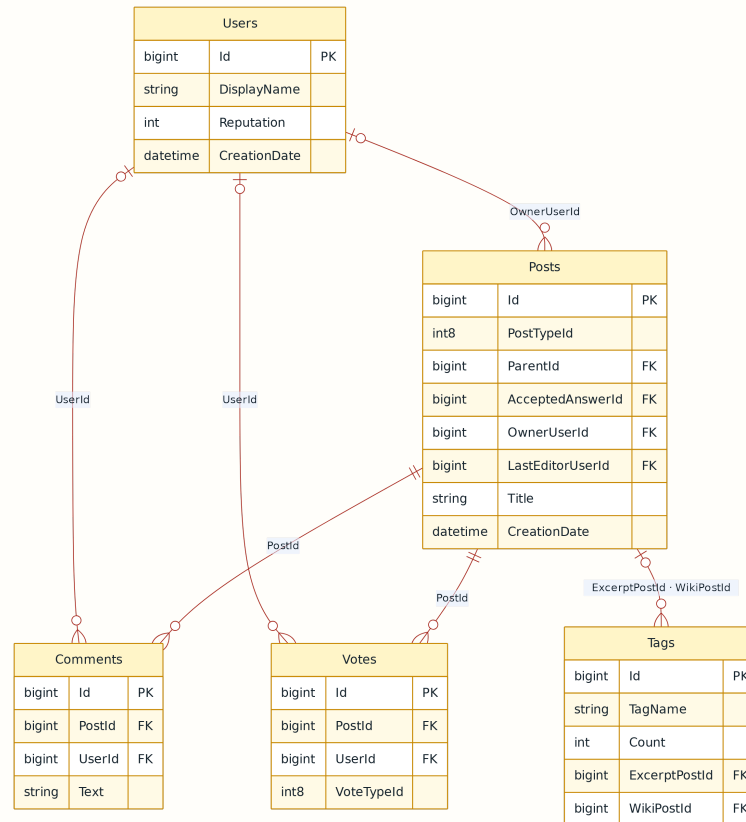
- **Posts:** preguntas y respuestas; `PostTypeId` distingue ambos tipos y `ParentId` enlaza una respuesta con su pregunta
- **Users:** usuarios y reputación
- **Comments** y **Votes:** actividad relacionada con un *post* y, cuando procede, un usuario
- **Tags** y **PostTags:** relación muchos-a-muchos entre preguntas y etiquetas
- En algunas instantáneas existe además una columna de etiquetas textual por motivos de rendimiento: es un ejemplo de desnormalización

Diagrama del esquema



El subconjunto que usaremos en prácticas

Las cinco tablas publicadas, con sus claves ajenas reales.



Lo que el diagrama no dice

- `Posts` se referencia a sí misma dos veces: `ParentId` enlaza una respuesta con su pregunta y `AcceptedAnswerId` la pregunta con su respuesta aceptada. Aparecen como claves ajenas, pero no se dibujan para no enredar el diagrama
- Casi todas las claves ajenas son **opcionales**: un *post* puede no tener usuario propietario porque la cuenta se borró, y un voto puede ser anónimo
- **Que una clave ajena esté declarada no significa que se cumpla**: este *dump* contiene votos y comentarios que apuntan a publicaciones ya borradas. Habrá que decidir explícitamente qué hacer con ellos antes de activar las restricciones
- El diagrama no está dibujado a mano: se genera a partir de los metadatos del propio esquema Parquet, los mismos que en la sesión 0 generaban el `CREATE TABLE`

SELECT: forma general

```
SELECT lista_de_columnas  
FROM lista_de_tablas  
WHERE condicion;
```

Primero se seleccionan las filas que cumplen la condición y después se proyectan las columnas solicitadas.

Elegir columnas

```
SELECT * FROM Tags;
```

```
SELECT Id, Title FROM Posts;
```

- `*` es útil para explorar, pero en consultas reales conviene pedir solo las columnas necesarias
- Las comillas de identificadores no son portables: evitamos depender de ellas usando nombres sencillos y sin palabras reservadas

DISTINCT, alias y LIMIT

```
SELECT DISTINCT PostTypeId FROM Posts;
```

```
SELECT DisplayName AS nombre FROM Users;
```

```
SELECT DisplayName FROM Users LIMIT 2;
```

- **DISTINCT** elimina duplicados del resultado
- **AS** nombra columnas o expresiones
- **LIMIT** facilita la exploración; no sustituye a un orden explícito

Ordenar resultados

```
SELECT Id, DisplayName, Reputation  
FROM Users  
ORDER BY Reputation DESC, Id ASC;
```

Sin `ORDER BY`, SQL **no garantiza** ningún orden de salida.

Selección y proyección

Álgebra relacional	SQL
Selección de filas: $\sigma_c(R)$	<code>SELECT * FROM R WHERE c</code>
Proyección de columnas: $\pi_{a, b}(R)$	<code>SELECT a, b FROM R</code>

```
SELECT Id, DisplayName
FROM Users
WHERE Reputation > 100;
```

El optimizador puede adelantar proyecciones, pero debe conservar las columnas necesarias para evaluar los filtros y las uniones.

Filtrar con WHERE

```
SELECT Id, DisplayName, Reputation
FROM Users
WHERE DisplayName = 'Elenasys';
```

Operador	Uso
= , <> , < , >=	Comparaciones
BETWEEN ... AND ...	Rango inclusivo
IN (...)	Pertenencia a una lista
LIKE	Patrón de texto
NOT , AND , OR	Composición lógica

Patrones y condiciones compuestas

```
SELECT DisplayName
FROM Users
WHERE Reputation > 100
      AND Location LIKE '%Spain%';
```

```
SELECT DisplayName
FROM Users
WHERE DisplayName LIKE '%sys%';
```

AND tiene mayor prioridad que **OR** ; usar paréntesis hace la intención inequívoca.

Expresiones y alias

```
SELECT DisplayName,  
       UpVotes + DownVotes AS votos_totales  
FROM Users  
WHERE UpVotes + DownVotes > 10;
```

El alias de `SELECT` no se puede usar de forma portable en `WHERE` : en esta fase de la consulta todavía no existe. Para reutilizarlo, se usa una subconsulta, una CTE o se repite la expresión.

NULL no es un valor ordinario

- NULL significa desconocido, ausente o no aplicable; no equivale a 0 ni a una cadena vacía
- col = NULL no encuentra filas: se usa col IS NULL o col IS NOT NULL
- Las comparaciones con NULL producen «desconocido», que WHERE no conserva

```
SELECT Id, ParentId
FROM Posts
WHERE ParentId IS NULL;
```

Funciones de agregación

```
SELECT AVG(Score) AS puntuacion_media,  
       MAX(Score) AS puntuacion_maxima,  
       COUNT(*) AS numero_posts  
FROM Posts;
```

- `COUNT(*)` cuenta filas; `COUNT(columna)` solo cuenta valores no nulos
- `AVG`, `SUM`, `MIN` y `MAX` ignoran los valores nulos
- `COUNT(DISTINCT OwnerUserId)` cuenta valores distintos no nulos

Funciones: núcleo y dialectos

Familia	Ejemplos portables o frecuentes
Numéricas	ABS , CEIL , POWER , SQRT
Cadenas	LOWER , UPPER , TRIM , SUBSTR , REPLACE
Control de nulos	COALESCE , NULLIF
Condicionales	CASE WHEN ... THEN ... ELSE ... END
Tipos	CAST(expr AS tipo)

CONCAT , funciones de desviación típica, aleatoriedad y fechas presentan diferencias entre motores; el notebook especifica las variantes necesarias.

CASE y COALESCE

```
SELECT DisplayName,  
       CASE  
         WHEN Reputation > 100 THEN 'Veterano'  
         ELSE 'Rookie'  
       END AS tipo,  
       COALESCE(UpVotes, 0) AS votos_positivos  
FROM Users;
```

COALESCE(a, b, ...) devuelve el primer argumento que no es nulo.

Fechas: filtrar sin perder el índice

Evitar, cuando se busca rendimiento:

```
WHERE YEAR(CreationDate) = 2025
```

Preferir un rango semicerrado:

```
WHERE CreationDate >= '2025-01-01'  
      AND CreationDate < '2026-01-01'
```

Las funciones y literales de fechas dependen del motor; el principio de no envolver la columna del filtro es general.

Consultar varias tablas

Producto cartesiano, `JOIN`, agrupación y subconsultas.

Producto cartesiano y equi-join

El producto cartesiano combina cada fila de `R1` con cada fila de `R2`. Rara vez es el resultado final deseado.

```
R1 ⋈Uid_r=Uid R2 =  $\sigma_{Uid_r=Uid} (R1 \times R2)$ 
```

Un **equi-join** conserva las combinaciones cuyas claves relacionadas coinciden.

JOIN explícito

```
SELECT c.Id, p.Title  
FROM Comments AS c  
JOIN Posts AS p ON p.Id = c.PostId  
WHERE p.Score > 10;
```

- El `JOIN ... ON` hace visible la condición de unión
- Evitamos la sintaxis antigua con tablas separadas por comas: es más fácil olvidar una condición y crear un producto cartesiano accidental

Preguntas y quien las hizo

```
SELECT u.DisplayName, p.Title
FROM Posts AS p
JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 1;
```

La misma idea se reutiliza para enlazar comentarios, votos, etiquetas y el historial con los *posts*.

LEFT JOIN: conservar filas sin coincidencia

```
SELECT p.Id, COUNT(a.Id) AS respuestas
FROM Posts AS p
LEFT JOIN Posts AS a
  ON a.ParentId = p.Id
  AND a.PostTypeId = 2
WHERE p.PostTypeId = 1
GROUP BY p.Id;
```

Las preguntas sin respuestas siguen apareciendo, con 0 respuestas.
Con un JOIN normal desaparecerían.

GROUP BY y HAVING

```
SELECT OwnerUserId, PostTypeId, COUNT(*) AS numero_posts
FROM Posts
GROUP BY OwnerUserId, PostTypeId
HAVING COUNT(*) > 10
ORDER BY OwnerUserId ASC, numero_posts DESC;
```

- WHERE filtra filas **antes** de agrupar
- HAVING filtra grupos **después** de agregar
- En SQL estricto, cada expresión seleccionada debe estar agrupada, agregada o ser funcionalmente dependiente de las columnas agrupadas

Agregación con nombres

```
SELECT u.DisplayName, COUNT(*) AS respuestas
FROM Posts AS p
JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 2
GROUP BY u.Id, u.DisplayName
HAVING COUNT(*) > 100
ORDER BY respuestas DESC;
```

La consulta explica una pregunta concreta: «¿qué usuarios han aportado muchas respuestas?».

Tablas como conjuntos

```
SELECT UserId FROM Comments  
UNION  
SELECT OwnerUserId FROM Posts;
```

- `UNION` elimina duplicados; `UNION ALL` los conserva
- `INTERSECT` y `EXCEPT` están disponibles en muchos motores, con diferencias de versión
- `IN` expresa pertenencia a un conjunto; con `NULL` hay que revisar cuidadosamente su semántica

Subconsultas y tablas derivadas

```
SELECT u.DisplayName, c.numero_comentarios
FROM Users AS u
JOIN (
    SELECT UserId, COUNT(*) AS numero_comentarios
    FROM Comments
    GROUP BY UserId
) AS c ON c.UserId = u.Id;
```

Una subconsulta en `FROM` produce una tabla derivada que se puede unir y consultar como las demás.

Consultas correlacionadas

```
SELECT u.Id, u.DisplayName
FROM Users AS u
WHERE 100 < (
    SELECT COUNT(*)
    FROM Posts AS p
    WHERE p.OwnerUserId = u.Id
    AND p.PostTypeId = 1
);
```

La subconsulta depende de la fila externa. Un optimizador puede transformarla, pero debemos comprobar el plan y los índices antes de asumir que tendrá un buen coste.

Promedio de respuestas: una sutileza de **NULL**

```
SELECT AVG(COALESCE(a.respuestas, 0)) AS media_respuestas
FROM Posts AS p
LEFT JOIN (
    SELECT ParentId, COUNT(*) AS respuestas
    FROM Posts
    WHERE PostTypeId = 2
    GROUP BY ParentId
) AS a ON a.ParentId = p.Id
WHERE p.PostTypeId = 1;
```

Sin **COALESCE**, **AVG** ignora las preguntas sin respuesta y la media queda sesgada al alza.

Consultas correctas y eficientes

El mismo resultado lógico puede requerir cantidades de trabajo muy distintas.

Leer un plan de MySQL

```
EXPLAIN
SELECT PostId
FROM Votes
WHERE PostId NOT IN (SELECT Id FROM Posts);
```

Empezaremos por preguntas simples:

- ¿Se recorre toda la tabla o se usa un índice?
- ¿Cuántas filas estima el optimizador?
- ¿Qué unión y qué orden de acceso ha elegido?

`EXPLAIN ANALYZE` añade medidas reales y ejecuta la consulta.

Ideas que conectan con BDGE

- Elegir solo las columnas necesarias evita transferencia y lectura innecesarias
- Filtrar pronto permite *predicate pushdown* en motores columnares y ficheros Parquet
- Los índices aceleran ciertos accesos, pero aumentan el coste de inserciones y actualizaciones
- Las estadísticas, la distribución de los datos y la cardinalidad determinan el plan elegido
- La desnormalización puede acelerar lecturas, a cambio de consistencia y coste de actualización

Para continuar

- Definir esquemas con DDL y restricciones
- Insertar, actualizar y borrar datos de forma segura
- Diseñar índices y contrastar planes con `EXPLAIN`
- Trabajar con JSON y decidir qué parte debe seguir siendo relacional